REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA
LA EDUCACIÓN UNIVERSITARIA
INSTITUTO UNIVERSITARIO DE TECNOLOGÍA
"ELÍAS CALIXTO POMPA"
INFORMÁTICA
ANÁLISIS Y DISEÑO DE SISTEMAS II

SISTEMA DE CONTROL DE ASISTENCIA MEDIANTE CÓDIGOS QR

Facilitador: William Hernández

Integrantes:

Fernando Rodríguez C.I: 31.853.086

Daniel Dias C.I: 30.937.908

Guatire, 01 de Agosto de 2026

Índice general

CAPÍTULO I

HERRAMIENTAS DE PROGRAMACIÓN Y CODIFICACIÓN DE LOS PROGRAMAS

1.1. Herramientas de Programación

Para la construcción del Sistema de Control de Asistencia mediante Códigos QR se emplearon las siguientes herramientas:

PHP 8.4: Lenguaje de programación del lado del servidor utilizado para toda la lógica del sistema. Se eligió PHP por su amplia compatibilidad con servidores web, su madurez y el soporte nativo para conexiones seguras a bases de datos mediante PDO (*PHP Data Objects*).

MariaDB 11.8: Sistema de gestión de bases de datos relacional utilizado para almacenar la información del sistema: usuarios, cursos, inscripciones, asistencias y registros de auditoría. Se utilizó el motor InnoDB para garantizar la integridad referencial mediante claves foráneas.

HTML5, CSS3 y JavaScript: Tecnologías del lado del cliente para la construcción de las interfaces de usuario. Se optó por JavaScript vanilla (sin frameworks) para minimizar dependencias externas y garantizar compatibilidad.

DDEV: Entorno de desarrollo basado en Docker que permite ejecutar el sistema localmente con PHP 8.4, MariaDB 11.8 y servidor nginx, replicando fielmente el entorno de producción.

Visual Studio Code: Editor de código fuente utilizado durante el desarrollo, con extensiones para PHP, SQL y control de versiones.

Git: Sistema de control de versiones para el seguimiento de cambios y la gestión del código fuente del proyecto.

1.2. Fragmento de Codificación

A continuación se presenta un fragmento representativo del código implementado. Se trata del endpoint encargado de validar el escaneo del código QR por parte del estudiante, el cual constituye una de las funcionalidades medulares del sistema:

```
<?php
require_once '../includes/config.php';
require_once '../includes/session.php';
require_once '../includes/functions.php';

verificarAutenticacion();
validarRol('estudiante');

$usuario_id = $_SESSION['usuario_id'];
$codigo_qr = limpiarInput($_POST['codigo_qr'] ?? '');

if (empty($codigo_qr)) {
    http_response_code(400);
    echo json_encode(['error' => 'Codigo QR requerido']);
    exit;
}

try {
    $stmt = $pdo->prepare(
        SELECT sq.id, sq.curso_id, sq.expira_en,
               sq.profesor_id, sq.latitud, sq.longitud,
               sq.radio_permitido, i.id AS inscripcion_id
        FROM sesiones_qr sq
        JOIN inscripciones i ON i.curso_id = sq.curso_id
            AND i.usuario_id = ?
            AND i.periodo = (SELECT codigo FROM periodos
                             WHERE activo = 1 LIMIT 1)
        WHERE sq.codigo_qr = ? AND sq.expira_en > NOW()
        LIMIT 1
```

```
30      );
31      $stmt->execute([$usuario_id, $codigo_qr]);
32      $sesion = $stmt->fetch();
33
34      if (!$sesion) {
35          http_response_code(404);
36          echo json_encode(['error' => 'QR invalido o expirado']);
37          exit;
38      }
39
40      $semana = obtenerSemanaActual();
41      $stmt = $pdo->prepare(
42          INSERT INTO asistencias (inscripcion_id, semana, fecha, estado)
43          VALUES (?, ?, CURDATE(), 'presente')
44          ON DUPLICATE KEY UPDATE estado = 'presente'
45      );
46      $stmt->execute([$sesion['inscripcion_id'], $semana]);
47
48      echo json_encode(['exito' => 'Asistencia registrada correctamente']);
49
50  } catch (PDOException $e) {
51      http_response_code(500);
52      echo json_encode(['error' => 'Error del servidor']);
53      registrarAuditoria($pdo, $usuario_id,
54          'Error al escanear QR', $e->getMessage());
55  }
56  ?>
```

Listing 1.1: Fragmento del endpoint de escaneo QR (`estudiante/escanear_qr.php`)

El fragmento anterior muestra la lógica de validación del código QR: verifica que el estudiante esté autenticado, que el QR exista y no haya expirado, que el estudiante se encuentre inscrito en el curso correspondiente, y finalmente registra la asistencia en la base de datos. Se emplean sentencias preparadas PDO para evitar inyección SQL y se

utiliza el formato JSON para las respuestas.

CAPÍTULO II

DESCRIPCIÓN Y PRESENTACIÓN DE LAS INTERFACES DEL SISTEMA

El sistema cuenta con interfaces diferenciadas para cada rol de usuario, diseñadas bajo los principios de claridad, eficiencia y adaptabilidad a dispositivos móviles. A continuación se presentan y describen las principales pantallas del sistema.

2.1. Interfaces del Profesor

El profesor dispone de un panel de control desde el cual gestiona la generación de códigos QR, consulta las asistencias en tiempo real y configura los parámetros de la sesión.

- **Generación de QR:** Pantalla principal donde el profesor selecciona el curso y la semana, y el sistema genera un código QR único con expiración configurable. El QR se autoregenera automáticamente para impedir reutilizaciones.
- **Panel de asistencias:** Muestra en tiempo real el listado de estudiantes inscritos con su estado de asistencia (presente/ausente) para la sesión activa.
- **Ajustes de configuración:** Permite al profesor definir el tiempo de expiración del QR (1–60 minutos) y el intervalo de rotación automática (10–300 segundos).

2.2. Interfaces del Estudiante

El estudiante accede al sistema desde su dispositivo móvil para registrar su asistencia y consultar su historial.

- **Pantalla de inicio:** Muestra las materias en las que el estudiante se encuentra inscrito durante el periodo activo, con acceso directo al módulo de escaneo.

- **Escaneo QR:** Activa la cámara del dispositivo para capturar el código QR proyectado por el profesor. Una vez escaneado, el sistema valida el QR y registra la asistencia automáticamente.
- **Historial de asistencia:** Presenta el registro cronológico de asistencias del estudiante, permitiéndole verificar su cumplimiento en cada materia.

2.3. Interfaces del Administrador

El panel de administración centraliza la gestión del sistema. Incluye módulos para la administración de usuarios, cursos, periodos académicos y la visualización de reportes de ausentismo con alertas automáticas para casos críticos.

2.3.1. Panel principal

Ofrece un resumen general del sistema con estadísticas de asistencia, usuarios activos y alertas importantes.

2.3.2. Gestión de entidades

Permite al administrador crear, modificar y eliminar usuarios, cursos, carreras y periodos académicos.

2.3.3. Reportes y datos

Genera reportes detallados de asistencia filtrados por curso, semana o estudiante, con opción de exportar a formato CSV.

2.3.4. Seguridad y accesos

Gestiona los permisos de usuarios, roles y auditoría de accesos al sistema.

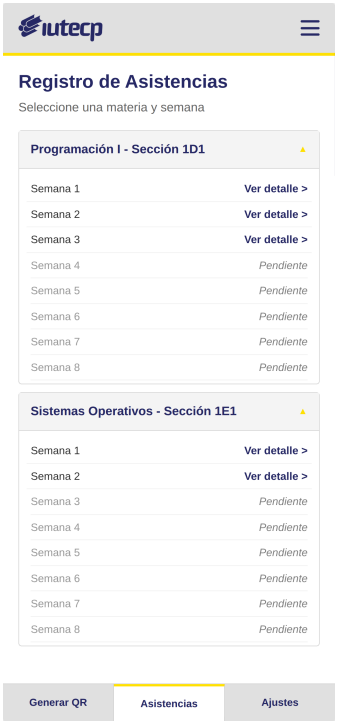
2.4. Galeria de Interfaces

A continuación se presentan las capturas de las principales interfaces del sistema.

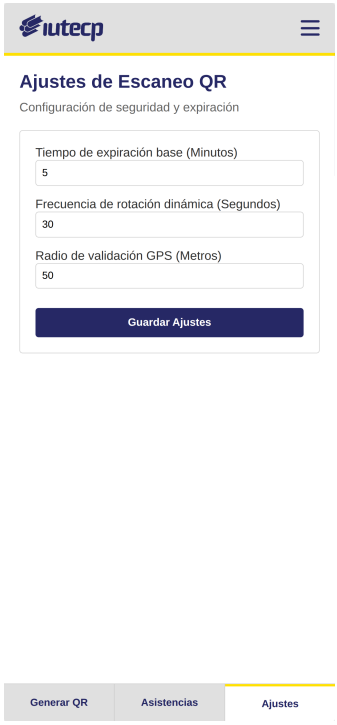
Interfaces del Profesor y Estudiante



Profesor: Generación QR



Profesor: Asistencias



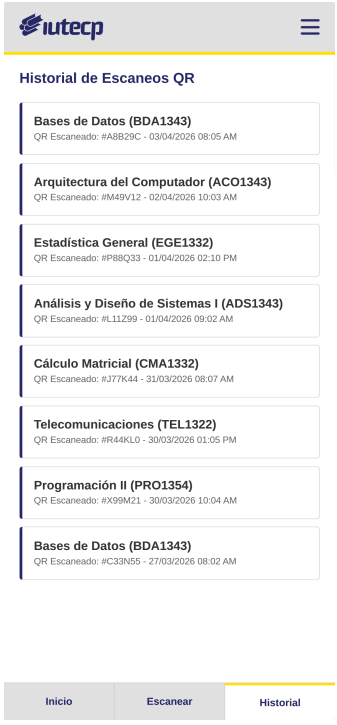
Profesor: Ajustes



Estudiante: Inicio



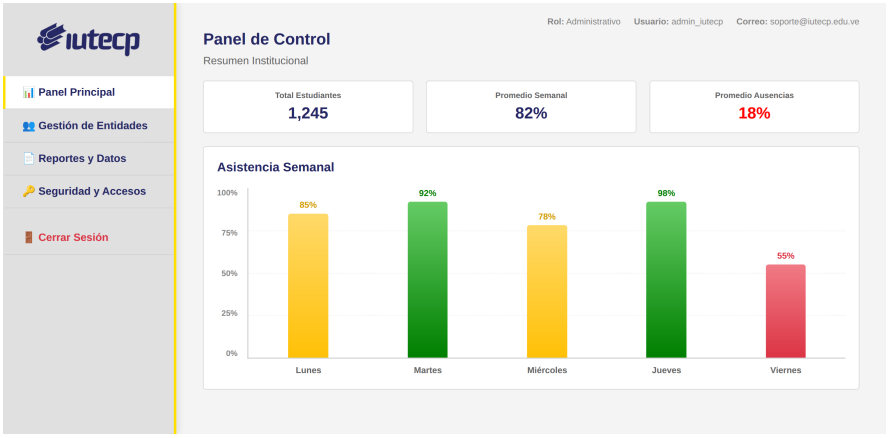
Estudiante: Escaneo QR



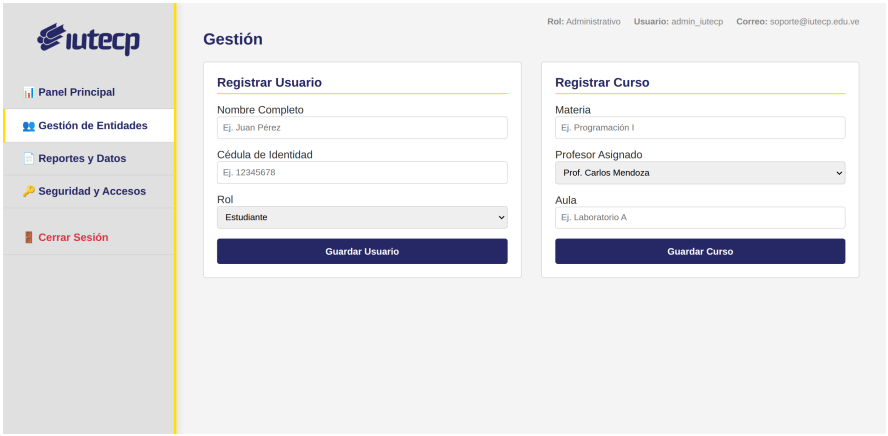
Estudiante: Historial

Interfaces del Administrador

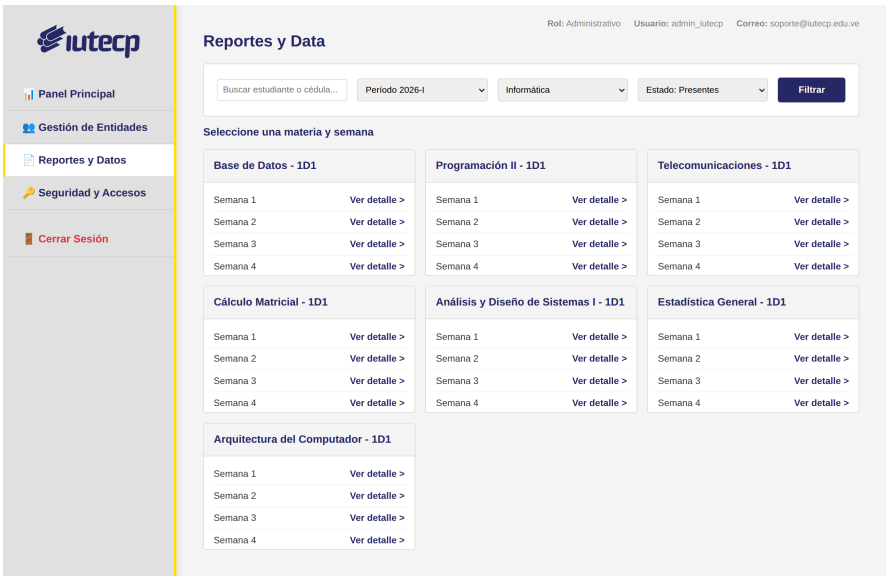
Las siguientes capturas muestran las interfaces del panel de administrador, diseñadas en formato landscape para mejor visualización:



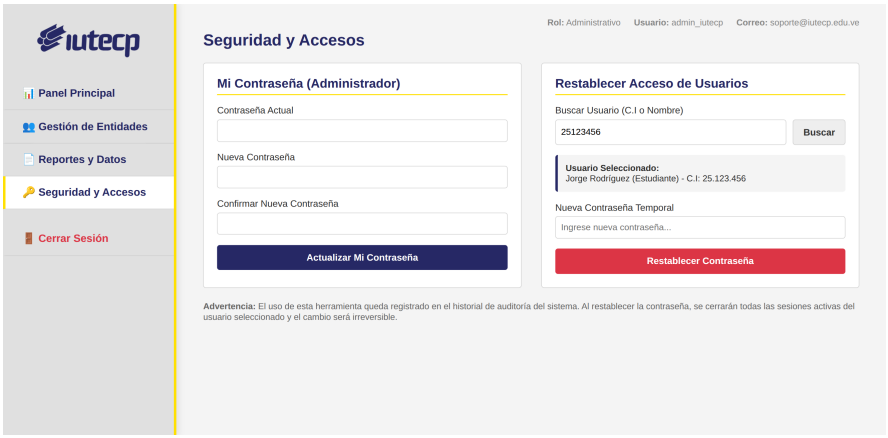
Panel principal



Gestión de entidades



Reportes y datos



Seguridad y accesos

CAPÍTULO III

CREACIÓN DE BASE DE DATOS Y LIBRERÍAS DE PROGRAMAS

3.1. Base de Datos

La base de datos del sistema, denominada `qr_iutecp`, fue diseñada bajo el modelo relacional utilizando MariaDB 11.8 con motor InnoDB. Está compuesta por trece tablas que en conjunto permiten gestionar usuarios, cursos, inscripciones, asistencias, sesiones QR, periodos académicos y auditoría.

3.1.1. Tablas del Sistema

A continuación se describen las principales tablas que conforman la base de datos:

- `usuarios`: Almacena los datos de todos los usuarios del sistema (administradores, profesores y estudiantes). Incluye campos como cédula (único), nombre de usuario (único), contraseña hasheada, correo electrónico, rol y estado (activo/inactivo).

- `carreras`: Contiene el catálogo de carreras ofrecidas por la institución (Informática, Administración, Contaduría).

- `cursos`: Registra las materias asociadas a cada carrera, con su código, nombre, nivel, unidades crédito y sección.

- `inscripciones`: Relaciona estudiantes con los cursos en los que están inscritos para un periodo académico específico.

- `asistencias`: Registra el estado de asistencia (presente, ausente o justificado) de cada estudiante por semana y fecha, con restricción de unicidad por inscripción-semana-fecha.

- `sesiones_qr`: Almacena los códigos QR generados por los profesores, incluyendo el código único, la fecha de expiración, las coordenadas GPS y el radio permitido.
- `configuracion_profesor`: Guarda la configuración personalizada de cada profesor: tiempo de expiración del QR, intervalo de rotación y radio GPS.
- `periodos`: Define los periodos académicos con sus fechas de inicio y fin, incluyendo soporte para estudiantes de nuevo ingreso.
- `auditoria`: Registra todas las acciones relevantes del sistema con identificador del usuario, tipo de acción, descripción, dirección IP y marca de tiempo.
- `prerrequisitos`: Define las dependencias entre cursos para validar la disponibilidad de materias según el avance del estudiante.

3.1.2. Esquema Relacional

A continuación se presenta el script de creación de las tablas principales, ejecutado para generar la estructura de la base de datos:

```
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    cedula VARCHAR(20) NOT NULL UNIQUE,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    rol ENUM('admin', 'profesor', 'estudiante') NOT NULL,
    carrera_id INT,
    activo TINYINT(1) DEFAULT 1,
    password_temporal TINYINT(1) DEFAULT 1,
    FOREIGN KEY (carrera_id) REFERENCES carreras(id)
);

CREATE TABLE inscripciones (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT NOT NULL,
    curso_id INT NOT NULL,
    periodo VARCHAR(10) NOT NULL,
    seccion VARCHAR(10),
    UNIQUE KEY (usuario_id, curso_id, periodo),
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
    FOREIGN KEY (curso_id) REFERENCES cursos(id)
);

CREATE TABLE asistencias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    inscripcion_id INT NOT NULL,
    semana INT NOT NULL,
    fecha DATE NOT NULL,
    estado ENUM('presente', 'ausente', 'justificado') DEFAULT 'ausente',
    UNIQUE KEY (inscripcion_id, semana, fecha),
    FOREIGN KEY (inscripcion_id) REFERENCES inscripciones(id)
);

CREATE TABLE sesiones_qr (
    id INT AUTO_INCREMENT PRIMARY KEY,
    profesor_id INT NOT NULL,
    curso_id INT NOT NULL,
    semana INT NOT NULL,
    codigo_qr VARCHAR(64) NOT NULL UNIQUE,
    expira_en DATETIME NOT NULL,
    FOREIGN KEY (profesor_id) REFERENCES usuarios(id),
    FOREIGN KEY (curso_id) REFERENCES cursos(id)
);
```

Listing 3.1: Fragmento del esquema de base de datos (`sql/schema.sql`)

3.2. Librerías de Programas

El sistema organiza su código en librerías reutilizables ubicadas en el directorio `includes/`, las cuales centralizan la lógica común yevitan la duplicación de código. A continuación se describen las principales:

Configuración y Conexión (`config.php`)

Este archivo establece la conexión a la base de datos mediante PDO (*PHP Data Objects*), configurando el manejo de errores en modo excepción y el juego de caracteres UTF-8. La conexión se almacena en la variable $pdo, utilizada por todos los módulos del sistema.

Manejo de Sesiones (`session.php`)

Contiene las funciones de autenticación y control de acceso:

- `estaAutenticado()`: Verifica si el usuario ha iniciado sesión.
- `redirigirSegunRol()`: Redirige al usuario al panel correspondiente según su rol.
- `registrarAuditoria()`: Inserta registros en la tabla de auditoría para rastrear las acciones de los usuarios.

Funciones Utilitarias (`functions.php`)

Agrupa funciones de uso general:

- `generarPasswordTemporal()`: Genera contraseñas aleatorias seguras para nuevos usuarios.
- `limpiarInput()`: Sanitiza datos de entrada para prevenir XSS.
- `generarUsername()`: Crea nombres de usuario automáticos basados en el nombre y apellido.

Lógica Académica (`academico.php`)

Implementa la lógica relacionada con la gestión académica:

- `obtenerPeriodoActivo()`: Determina el periodo académico vigente.
- `obtenerSemanaActual()`: Calcula la semana académica actual según la fecha de inicio del periodo.
- `verificarPrerrequisitos()`: Valida si un estudiante cumple con los prerrequisitos necesarios para cursar una materia.
- `obtenerMateriasDisponibles()`: Retorna las materias que un estudiante puede cursar según su avance en el pensum.

Generación de Códigos QR (`phpqrcode/phpqrcode.php`)

Se utiliza la biblioteca *PHP QR Code* (versión 1.1.4) para la generación de códigos QR. Esta librería, bajo licencia LGPL 3, proporciona funciones para crear códigos QR a partir de texto, URLs o datos estructurados. El archivo incluido es una versión fusionada que no requiere archivos externos y viene configurada con caché deshabilitada y una búsqueda de más rápida. El sistema emplea esta biblioteca para generar códigos QR únicosen para cada sesión de clase, con expiración configurable y soporte para geolocalización.

REFERENCIAS

1. PHP Group. (2024). *PHP Manual*. https://www.php.net/manual
2. MariaDB Foundation. (2024). *MariaDB Knowledge Base*. https://mariadb.com/kb
3. Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico* (7.a ed.). McGraw-Hill.
4. Sommerville, I. (2011). *Ingeniería de software* (9.a ed.). Pearson.
5. W3C. (2023). *HTML Living Standard*. https://html.spec.whatwg.org
6. MDN Web Docs. (2024). *JavaScript Guide*. https://developer.mozilla.org
7. Dominik Dzienia. (2010). *PHP QR Code Encoder*. http://phpqrcode.sourceforge.net